WEB SYSTEMS DESIGN – PROJECT 3

VISTULA

UNIVERSITY

Name: Amjad Massaoud

Student ID: 78709

Course: Web Systems Design

Semester: 6th semester 57DPH

Date: 30.08.2026

Step 1 - Configure database connection 3

Step 2 – Create PostgreSQL Database Manually 3

Step 3 – Create Migration for Tasks Table 3

Step 4 – Edit Migration Structure 4

Step 5 – Run Migration 4

Step 6 – Create Task Model 4

Step 7 – Configure Fillable Fields 4

Step 8 – Create API Controller for CRUD 5

Step 9 & 10 & 11 – Implement CRUD Methods/Register API Routes and Start Server 5

📄 **Additional Explanations** 6

- What is CRUD? 6
- Why Validation is Important? 6
- Why Album Number is Required? 7
- Difference Between HTTP Methods 7
- Why HTTP Status Codes are Important? 7
- Persistent Storage in Backend Systems 7

Step 12 – commit changes to git 14

Step 1 - Configure database connection

I opened the backend/.env file and updated the database configuration to use PostgreSQL. I replaced the default database settings with my own PostgreSQL credentials.

.env

```
22  DB_CONNECTION=pgsql
23  DB_HOST=127.0.0.1
24  DB_PORT=5432
25  DB_DATABASE=wsd_tasks
26  DB_USERNAME=postgres
27  DB_PASSWORD=
28
```

Step 2 – Create PostgreSQL Database Manually

I opened PostgreSQL using pgAdmin and manually created a new database named wsd_tasks using the SQL command `CREATE DATABASE wsd_tasks;`.

```
jad@fedora-2:~/uni-project/project3/backend$ php artisan config:clear
INFO Configuration cache cleared successfully.

jad@fedora-2:~/uni-project/project3/backend$ php artisan migrate
INFO Preparing database.
Creating migration table ..................................... 9.22ms DONE
INFO Running migrations.
```

Database	Owner	Comment
codeguardian	postgres	
postgres	postgres	default administrative connection ...
wsd_tasks	postgres	

Step 3 – Create Migration for Tasks Table

I generated a new migration file for the tasks table by running the command `php artisan make:migration create_tasks_table` in the terminal.

Laravel automatically created a new migration file inside the database/migrations directory. This file is used to define the structure of the tasks table in the database.

```
jad@fedora-2:~/uni-project/project3/backend$ php artisan make:migration
create_tasks_table
INFO Migration [database/migrations/2026_08_30_210000_create_tasks_table.php] created
successfully.
```

Step 4 – Edit Migration Structure

I opened the newly created migration file and modified the Schema::create block to define the structure of the tasks table.

I added the following fields: **id, title, description, status, album_number, and timestamps**. The description field was set as nullable, and the status field was given a default value of todo.

Step 5 – Run Migration

I executed the command `php artisan migrate` to apply the migration and create the tasks table in the database.

After running the command, Laravel successfully created the table, and the terminal displayed a confirmation message indicating that the migration was completed.

To verify the result, I checked the database and confirmed that the tasks table was created with the correct structure.

```
jad@fedora-2:~/uni-project/project3/backend$ php artisan migrate
INFO Running migrations.
2026_08_30_210000_create_tasks_table ..............................  15.48ms DONE
```

Step 6 – Create Task Model

I generated the Task model using the Artisan command `php artisan make:model Task`.

```
jad@fedora-2:~/uni-project/project3/backend$ php artisan make:model Task
INFO Model [app/Models/Task.php] created successfully.
```

Step 7 – Configure Fillable Fields

I opened the app/Models/Task.php file and updated the model by defining the `$fillable` property. This property specifies which fields are allowed for mass assignment.

I included the fields **title, description, status, and album_number** to ensure that these values can be safely inserted into the database.

2026_08_30_210000_create_tasks_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('tasks', function (Blueprint $table) {
            $table->id();
            $table->string('title');
            $table->text('description')->nullable();
            $table->string('status')->default('todo');
            $table->string('album_number');
            $table->timestamps();
        });
    }
};


```

Step 8 – Create API Controller for CRUD

I generated a new API controller using the Artisan command `php artisan make:controller Api/TaskController`.

```
jad@fedora-2:~/uni-project/project3/backend$ php artisan make:controller
Api/TaskController
INFO Controller [app/Http/Controllers/Api/TaskController.php] created successfully.
```

Step 9 & 10 & 11 – Implement CRUD Methods/Register API Routes and Start Server

I opened the routes/api.php file and registered the API routes for the TaskController.

I implemented full CRUD functionality inside the TaskController following REST API principles and Laravel best practices.

The controller includes the following methods:

- **index()** → retrieves all tasks from the database and returns them as a JSON response with status 200 OK
- **store()** → validates incoming request data, creates a new task (including required album_number), and returns the created object with status 201 Created
- **show()** → retrieves a single task by its ID using find and returns it as JSON or 404 if not found
- **update()** → updates an existing task with validated data and returns the updated object with status 200 OK
- **destroy()** → deletes a selected task and returns an empty response with status 204 No Content
- **summary()** → custom helper endpoint returning aggregated task counts by status

I used Laravel validation to ensure required fields such as title and album_number are provided. All database operations were handled using Eloquent ORM instead of raw SQL.

The controller consistently returns JSON responses and uses appropriate HTTP status codes, ensuring compliance with REST API standards.

What is CRUD?

CRUD stands for:

- **Create** → adding new data (POST)
- **Read** → retrieving data (GET)
- **Update** → modifying existing data (PUT/PATCH)
- **Delete** → removing data (DELETE)

These operations form the foundation of most backend systems.

Why Validation is Important?

Validation ensures that only correct and safe data is stored in the database. It prevents errors, improves data integrity, and protects the application from invalid or malicious input.

Why Album Number is Required?

The album_number field is required to identify the student who created the task. It is mandatory for laboratory verification and ensures that each record can be linked to a specific user.

Difference Between HTTP Methods

- **GET** → retrieves data
- **POST** → creates new data
- **PUT/PATCH** → updates existing data
- **DELETE** → removes data

Each method represents a specific type of operation in REST APIs.

Why HTTP Status Codes are Important?

HTTP status codes provide information about the result of a request. They help developers understand whether an operation was successful or if an error occurred.

Examples:

- **200** → success
- **201** → resource created
- **204** → no content (deleted)
- **404** → resource not found
- **422** → validation error

Persistent Storage in Backend Systems

Persistent storage refers to saving data permanently in a database so that it remains available even after the application is restarted. In this project, PostgreSQL is used as a persistent storage system. When a task is created, updated, or deleted, the changes are stored directly in the database through Laravel's Eloquent ORM. This ensures that all data is permanently saved and can be retrieved later using API requests.

Task.php — backend > app > Models > Task.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Task extends Model
{
    protected $fillable = [
        'title',
        'description',
        'status',
        'album_number',
    ];
}
```

```
jad@fedora-2:~/uni-project/project3/backend$ php artisan route:list
GET|HEAD / ..........................................................
GET|HEAD api/tasks ................................ Api\TaskController@index
POST api/tasks ................................... Api\TaskController@store
GET|HEAD api/tasks/summary ....................... Api\TaskController@summary
GET|HEAD api/tasks/{id} ........................... Api\TaskController@show
PUT api/tasks/{id} ............................ Api\TaskController@update
DELETE api/tasks/{id} ......................... Api\TaskController@destroy
Showing [17] routes
```

api.php — backend > routes > api.php

```php
<?php

use App\Http\Controllers\Api\TaskController;
use Illuminate\Support\Facades\Route;

Route::get('/tasks', [TaskController::class, 'index']);
Route::post('/tasks', [TaskController::class, 'store']);
Route::get('/tasks/summary', [TaskController::class, 'summary']);
Route::get('/tasks/{id}', [TaskController::class, 'show']);
Route::put('/tasks/{id}', [TaskController::class, 'update']);
Route::delete('/tasks/{id}', [TaskController::class, 'destroy']);
```

Postman API Testing - GET Initial Tasks (Empty List)

WSD Project / Tasks API

GET http://127.0.0.1:8000/api/tasks

Body • Pretty • JSON — Status: 200 OK • Time: 120 ms • Size: 2 B

```
[ ]
```

Postman API Testing - POST Create Task

WSD Project / Tasks API

POST http://127.0.0.1:8000/api/tasks

```
{
  "title": "Homework Task",
  "description": "Laravel CRUD test",
  "status": "todo",
  "album_number": "78709"
}
```

Body • Pretty • JSON — Status: 201 Created • Time: 257 ms • Size: 417 B

```
{
    "title": "Homework Task",
    "description": "Laravel CRUD test",
    "status": "todo",
    "album_number": "78709",
    "updated_at": "2026-08-30T15:24:06.000000Z",
    "created_at": "2026-08-30T15:24:06.000000Z",
    "id": 1
}
```

Postman API Testing - PUT Update Task

WSD Project / Tasks API

PUT `http://127.0.0.1:8000/api/tasks/1`

```
{
  "title": "Homework Task Updated",
  "description": "Laravel CRUD test",
  "status": "in_progress",
  "album_number": "78709"
}
```

Body • Pretty • JSON — **Status: 200 OK • Time: 134 ms • Size: 412 B**


```
{
    "id": 1,
    "title": "Homework Task Updated",
    "description": "Laravel CRUD test",
    "status": "in_progress",
    "album_number": "78709",
    "created_at": "2026-08-30T15:02:07.000000Z",
    "updated_at": "2026-08-30T15:27:24.000000Z"
}
```

Postman API Testing - DELETE Task

WSD Project / Tasks API

DELETE http://127.0.0.1:8000/api/tasks/1

Body • Raw Status: 204 No Content • Time: 134 ms • Size: 197 B

<Empty response body>

Postman API Testing - GET Remaining Tasks List

WSD Project / Tasks API

GET http://127.0.0.1:8000/api/tasks

Body • Pretty • JSON Status: 200 OK • Time: 116 ms • Size: 591 B


```
[
    {
        "id": 2,
        "title": "Laboratory Task 2",
        "description": "Eloquent validation",
        "status": "todo",
        "album_number": "78709",
        "created_at": "2026-08-30T15:24:06.000000Z",
        "updated_at": "2026-08-30T15:24:06.000000Z"
    }
]
```

TaskController.php Source Code (Part 1)

TaskController.php backend > app > Http > Controllers > Api > TaskController.php

```php
<?php

declare(strict_types=1);

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Task;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

class TaskController extends Controller
{
    /**
     * GET /api/tasks
     */
    public function index(): JsonResponse
    {
        return response()->json(Task::all(), 200);
    }

    /**
     * POST /api/tasks
     */
    public function store(Request $request): JsonResponse
    {
        $validated = $request->validate([
            'title' => 'required|string|max:255',
            'description' => 'nullable|string',
            'status' => 'nullable|string',
            'album_number' => 'required|string',
        ]);

        if (!isset($validated['status'])) {
            $validated['status'] = 'todo';
        }

        $task = Task::create($validated);

        return response()->json($task, 201);
    }

    /**
     * GET /api/tasks/{id}
     */
    public function show(string|int $id): JsonResponse
    {
        $task = Task::find($id);

        if (!$task) {
            return response()->json(['message' => 'Task not found'], 404);
        }

        return response()->json($task, 200);
    }
```

TaskController.php Source Code (Part 2)

TaskController.php

```php
    /**
     * PUT /api/tasks/{id}
     */
    public function update(Request $request, string|int $id): JsonResponse
    {
        $task = Task::find($id);

        if (!$task) {
            return response()->json(['message' => 'Task not found'], 404);
        }

        $validated = $request->validate([
            'title' => 'sometimes|string|max:255',
            'description' => 'sometimes|nullable|string',
            'status' => 'sometimes|string',
            'album_number' => 'sometimes|string',
        ]);

        $task->update($validated);

        return response()->json($task, 200);
    }

    /**
     * DELETE /api/tasks/{id}
     */
    public function destroy(string|int $id): JsonResponse
    {
        $task = Task::find($id);

        if (!$task) {
            return response()->json(['message' => 'Task not found'], 404);
        }

        $task->delete();

        return response()->json(null, 204);
    }
}
```

Step 12 – commit changes to git

I staged all modified and newly created controller, model, migration, and route files, committed them with a descriptive message, and pushed the branch to the remote GitHub repository **Jadtales/uni-projects**.

```
jad@fedora-2:~/uni-project/project3/backend$ git add .
jad@fedora-2:~/uni-project/project3/backend$ git commit -m "Project 3: add PostgreSQL CRUD tasks API"
[main e6ae964] Project 3: add PostgreSQL CRUD tasks API
4 files changed, 151 insertions(+), 3 deletions(-)
jad@fedora-2:~/uni-project/project3/backend$ git push
To https://github.com/Jadtales/uni-projects.git
88c2b21..e6ae964 master -> master
```